



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

HTML (I parte)

Fabio Vitali

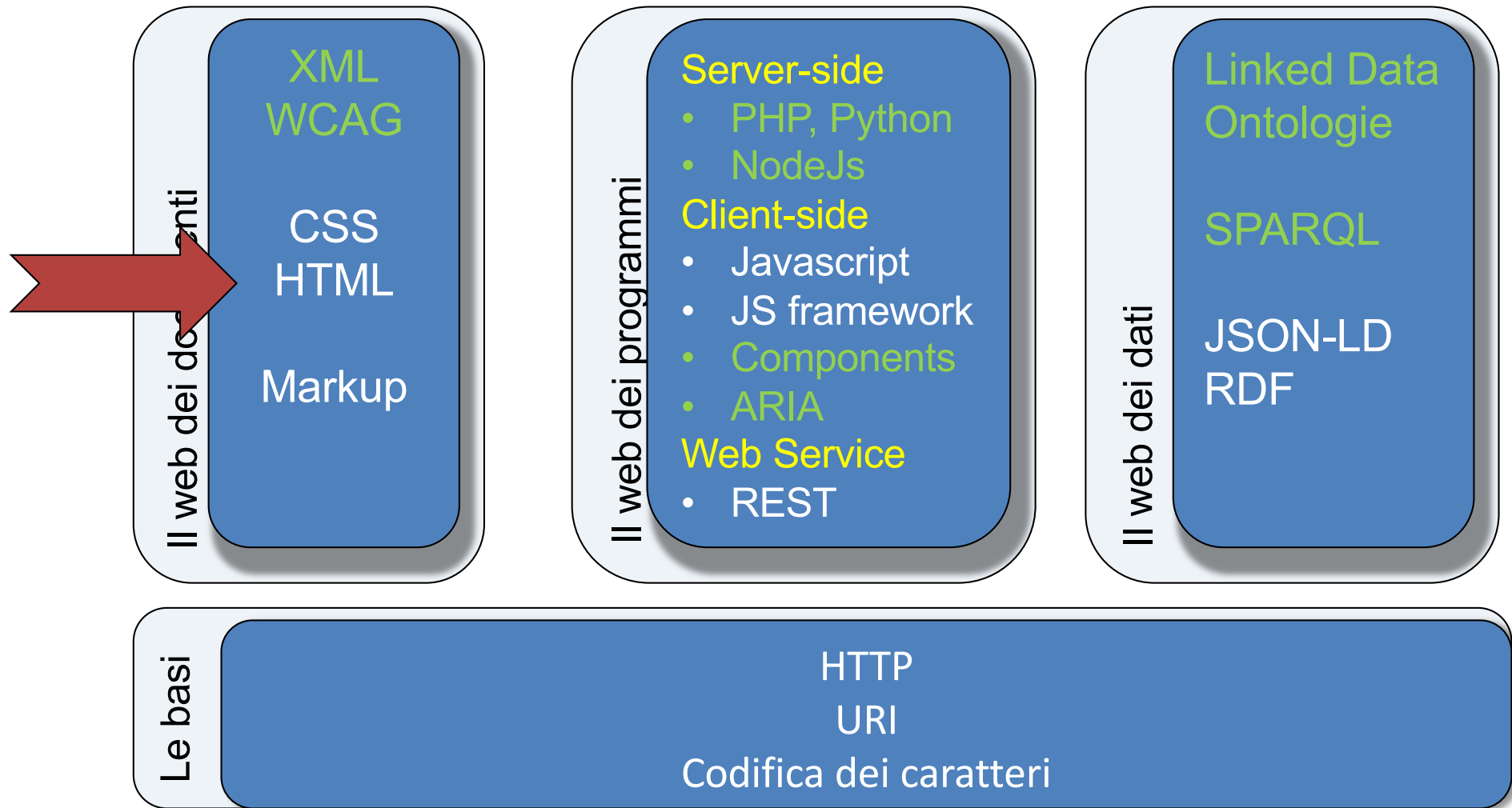
CdS in Informatica

Introduzione

Oggi esaminiamo in breve:

- La storia e le caratteristiche di HTML
- HTML (concetti base)

Argomenti delle lezioni



Contenuti per tutti
Contenuti solo per 9CFU



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Breve storia di HTML

Le browser wars

- 1994-2001: Netscape Navigator (il primo browser di grande successo) è in competizione con Microsoft Internet Explorer. Alla lunga vince Microsoft, Netscape fallisce e rilascia in Open Source il codice della versione 5 di Navigator che diventerà la base di Firefox. Ma il vero vincitore è il W3C che viene riconosciuto come tavolo neutrale e decisore ultimo sul futuro degli standard del Web
- 2005-2010: Molti attori in gioco: Firefox contro Chrome, Safari contro Chrome (e quindi Apple contro Google ma anche piattaforme mobili contro PC desktop) ma soprattutto WHAT WG contro W3C, quindi browser "professionali" contro tutti.
- 2019-*in corso*: I sistemi a componenti proposti per la prima volta da AngularJS creano tre grandi famiglie di librerie: Angular (senza JS) da Google, React e React Native da Facebook, e VueJs dalla comunità Open Source. I browser diventano progressivamente meno importanti, rispetto al modello di sviluppo di applicazioni web. Ancora non si vede un vincitore chiaro.



HTML oggi

HTML è diventato qualcosa di diverso rispetto a ciò che è stato per molti anni: da linguaggio di markup per documenti ipertestuali e multimediali ad un vero e proprio linguaggio per applicazioni

E' il World Wide Web che sta cambiando e l'attenzione è sempre più su interattività, dinamicità e manipolazione sofisticata (client-side) dei contenuti

Il browser non è più soltanto uno strumento di lettura (senza dimenticare che inizialmente era anche un editor!) ma diventa un motore generico per l'esecuzione di applicazioni

E HTML cambia sia nella semantica che nella sintassi, rendendo vani molti degli sforzi fatti dal W3C negli anni.

Anche perché c'è ancora confusione tra diverse versioni dello standard, proposte parallele e gli stessi gruppi di lavoro...



HTML 4

Il linguaggio HTML un tipo di documenti SGML (esiste un DTD di HTML, anzi più di uno) progettato per marcare documenti ipertestuali. Tramite HTML è possibile realizzare documenti con una semplice struttura, con aspetti grafici anche sofisticati, con testo, immagini, oggetti interattivi e link.

L'inizio della prima “guerra dei browser” (1994-98) ha portato alla creazione di numerosi elementi proprietari, estensioni, effetti sofisticati e caratteristiche puramente presentazionali

Dopo diverse versioni intermedie, HTML 4.0 (1997) “chiude i giochi” e aggiunge il supporto l’internazionalizzazione, per gli style sheet, per i frame, tabelle molto più ricche, etc. HTML 4.01 (1999) apporta modifiche minori

Da allora lo standard è rimasto sostanzialmente invariato fino al 2008.



Da HTML a “tag soup”

Molte pagine HTML tuttavia sono diventate “tag soup”, ossia un insieme di elementi non conformi allo standard. I browser infatti si sono preoccupati poco della correttezza sintattica o strutturale dei documenti HTML.

Questo significa che esistono differenze anche sensibili tra un documento HTML corretto e un documento HTML visualizzabile da un browser Web

Ma come fa il browser ad interpretare i documenti sintatticamente non corretti? Ognuno fa a modo suo!

Prima di WHATWG, infatti, non era mai stato standardizzato il modo in cui fare il parsing di questi documenti.

Inoltre per gestire sia le pagine conformi allo standard che quelle non compatibili (proliferate dopo la “guerra dei browser”), sono stati introdotti due modelli di rendering: quirks mode e strict mode



Quirks mode e strict mode

Molte pagine HTML 4 sono state scritte usando un modello scorretto e locale ad uno specifico browser. Queste stesse pagine non funzionano con i browser che implementano correttamente lo standard.

E' necessario capire se il browser dovesse interpretare una pagina secondo lo standard oppure no.

La soluzione è avere due stili di interpretazione di HTML e CSS:

- ***quirks mode***, compatibile con il passato e più permissivo
- ***strict mode***, compatibile con le specifiche ufficiali

Se la pagina non specifica niente, il browser adotta il modo compatibile (*quirks mode*), altrimenti se l'autore lo richiede esplicitamente attiva il modo restrittivo e corretto.

XHTML e poi HTML 5 cambiano le regole standardizzando le regole di parsing (anche e soprattutto in caso di errori)



XHTML 1.0

XHTML 1.0 è una Recommendation W3C del 2000. È una riformulazione di HTML 4 come un'applicazione di XML 1.0.

L'elenco e la semantica di elementi e attributi non è assolutamente cambiata rispetto a HTML 4.

I documenti debbono essere ben formati, in particolare l'annidamento deve essere corretto.

Altre differenze con HTML 4 (alcune):

- I nomi di elementi e attributi sono minuscoli (XML è case-sensitive)
- Il tag finale è obbligatorio per elementi non vuoti
- Gli elementi vuoti debbono seguire la sintassi XML
- I valori degli attributi debbono sempre avere le virgolette



(X)HTML 5

Nel 2004, Firefox e Opera proposero al W3C la riapertura del Working Group su HTML per lo sviluppo di nuove versioni del linguaggio. La proposta, che ignorava volutamente XHTML e la rigida sintassi di XML, venne bocciata dal W3C.

Venne così creata una comunità a partecipazione ristretta, chiamata WHATWG (Web Hypertext Application Technology Working Group), finanziata e supportata da Mozilla, Opera e Apple, che inizia a lavorare ad una versione “intermedia” di HTML, “Web Application 1.0” (WA1), meno ambiziosa di XHTML 2.0. Il chair, Ian Hickson, diventa poi un project manager a Google.

Nel 2007 il W3C ammise che queste modifiche avevano un impatto innegabile (*“he who ships working code wins”*), riaprì il working group con tutti i membri del WHATWG col fine di creare una nuova versione del linguaggio, (X)HTML 5. Ma non è finita qui.



(~~X~~)HTML 5

WA1 fa propria l'opinione da parte dei browser quando dicono che non è mai esistita una grammatica SGML per HTML, ma che il linguaggio è sempre stato

- O un'applicazione di XML (XHTML *)
- Oppure una tag soup in cui i browser accettano ogni sorta di “porcheria” e fanno il meglio che possono

WA1 è più interessato a mettere a posto le cose che non vanno piuttosto che lavorare su un linguaggio nuovo.

E il W3C deve adeguarsi, anche perché WA1 ha supporto da tutti i browser e di tutta l'industria delle applicazioni Web

“(X)HTML 5” diventa quindi solo “HTML 5” e successivamente solo “HTML”



HTML Living Standard

Improvvisamente, e senza informare il W3C, nel 2011 la versione di HTML gestita da WHAT WG diventa una specifica in sviluppo continuo (“living standard”):

Instabilità e continuo cambiamento diventano feature:

Because the specification is now a living document, we are today announcing two changes:

- *The HTML specification will henceforth just be known as "HTML".*
- *The WHATWG HTML spec can now be considered a "living standard".*

It's more mature than any version of the HTML specification to date, so it made no sense for us to keep referring to it as merely a draft.

Ian Hickson, chair del WHATWG, 19 gennaio 2011

Cambia completamente il modello di sviluppo del linguaggio che si allontana dall’approccio sistematico e democratico di evoluzione degli altri standard W3C. E’ la vittoria indiscussa dei produttori di browser: sia per le caratteristiche del linguaggio sia per il modo in cui è sviluppato.



La fine della divisione su HTML

Nel linguaggio esistevano due “anime” incompatibili, con obiettivi diversi:

- La componente W3C voleva “stabilizzare” le versioni di HTML5, per farle diventare Recommendation, con test di conformità e modello approvato con il processo W3C. Inoltre il W3C insiste che HTML indichi solo il linguaggio di markup.
- La componente WHATWG voleva continuare lo sviluppo continuo del linguaggio di markup e di molte tecnologie collegate, in maniera integrata e fortemente collegata alle implementazioni esistenti.

Nel 2011 i due gruppi si separano. Il W3C quindi pubblica un sottoinsieme delle specifiche di “HTML Living Standard” come “HTML5.x”

- Alcune feature sono omesse, altre pubblicate in specifiche separate (es. Canvas 2D)

Negli anni il W3C approva un HTML 5.1, un HTML 5.2 e un draft iniziale di HTML 5.3, progressivamente divergenti da HTML Living Standard. Non c'è però supporto dai browser.

Per questo motivo, nel maggio 2019 il W3C annuncia ufficialmente che le nuove release del linguaggio saranno in carico al WHATWG e che cesserà la produzione di un linguaggio diverso da quello. Anche il W3C adotta implicitamente il modello "Living Standard".



Ma poi quali sono tutte queste modifiche frequentissime?

2011 — wbr

2011 — time rimosso

2012 — time reinserito con datetime riveduto.

2013 — main

2013 — template (bozza dedicata).

2014 — Immagini responsive: <picture>, <source>, img[srcset|sizes].

2014 — data; discusso già nel 2011.

2014 — dialog (prima in Chromium 37, 2014, poi incluso in HTML 5.2, 2017).

2016 — aggiornamento a form (es. month, week, datetime-local) + **details/summary**.

2017 — rimozioni (keygen, menu/menuitem) e altre pulizie.

2018 — img[decoding].

2019 — Attributo globale enterkeyhint (adottato 2021).

2020 — loading="lazy" per /<iframe>.

2022 — inert (attributo globale); hidden="until-found".

2022 — fetchpriority per , <link>, <script>.

2023 — Declarative Shadow DOM: <template shadowrootmode="open|closed">

2023 — Import Maps: <script type="importmap"> (confluite nella HTML LS).

2024 — popover (attributo globale) + popovertarget

2025 — contenteditable="plaintext-only" (nuovo valore).



Peculiarità di HTML Living Standard

- Il linguaggio di markup è stato rivisto globalmente, introducendo nuovi elementi più specifici per usi e abitudini ormai consolidate (e.g., `<nav>`, `<section>` e `<article>` invece di `<div>`, `<audio>` e `<video>` invece di `<object>`, ecc.)
- L'uso di script per la modifica dinamica *in memory* del documento (DOM editing) è diventata parte integrante del linguaggio, e ne costituisce un elemento prevalente rispetto alla linearizzazione in stringa del markup HTML.
- Il doctype basato su SGML è definitivamente abbandonato, e quello basato su XML è solo facoltativo. La filosofia resta, ma HTML è ora sufficientemente importante di suo da non dover dipendere da una meta-sintassi terza per le sue caratteristiche.
- Oltre al markup, HTML definisce e descrive un certo numero di API per un uso del DOM più sofisticato di quanto descrivibile solo col markup.
- XHTML è una linearizzazione facoltativa di HTML basata sulla sintassi XML. Ha poche ma importanti differenze da HTML.





ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

HTML – le basi

HTML – le basi

Adesso parliamo di:

- La struttura di un documento HTML
- Elementi inline
- Elementi di blocco e di lista
- Elementi generici
- Elementi di struttura
- *Link ed immagini*
- *Tipi di dati*
- *Entità predefinite*
- *Peculiarità sintattiche*
- *Tabelle*
- *Form*
- *Il contenuto dell'elemento HEAD*



La struttura di un documento HTML

<!DOCTYPE html>: segnala il tipo di markup usato nel documento e la sua versione, con molte complessità e variazioni.

Case-insensitive, ma se si usa XHTML e XHTML 5, allora DOCTYPE è maiuscolo e tutti i tag sono minuscoli.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Document title</title>
  </head>
  <body>
    <p>Text of a paragraph</p>
  </body>
</html>
```

html: La radice dell'albero.

head: Informazioni globali sul documento

body: Il contenuto del documento



Un primo esempio di HTML

```
<!DOCTYPE html>
<html>
  <head>
    <title> Document title </title>
  </head>
  <body>
    <h1> Major Header </h1>
    <p>This is a complete paragraph of a document. I write
and write until I fill in several lines, since I want to see
how it wraps automatically. Surely not a very exciting
document.</p>
    <p>Did you expect <b>poetry</b>?</p>
    <p>Here you can see a paragraph <br>
      split by a &lt;br></p>
    <hr>
    <p> A list of important things to remember: </p>
    <ul>
      <li>Spaces, tabs and returns</li>
      <li>Document type declaration</li>
      <li>Document structure</li>
      <li>Nesting and closing tags</li>
    </ul>
  </body>
</html>
```



Un primo esempio di HTML

```
<!DOCTYPE html>
<html>
  <head>
    <title> Document title
  </head>
  <body>
    <h1> Major Header </h1>
    <p>This is a complete
and write until I fill in
how it wraps automatically
document.</p>
    <p>Did you expect <b>
    <p>Here you can see a
      split by a &lt;br>
    <hr>
    <p> A list of important
  <ul>
    <li>Spaces, tabs and
    <li>Document type de
    <li>Document structu
    <li>Nesting and clos
  </ul>
</body>
</html>
```

Major Header

This is a complete paragraph of a document. I write and write until I fill in several lines, since I want to see how it wraps automatically. Surely not a very exciting document.

Did you expect **poetry**?

Here you can see a paragraph split by a

A list of important things to remember:

- Spaces, tabs and returns
- Document type declaration
- Document structure
- Nesting and closing tags

Elementi inline (1)

Gli elementi *inline* (o di carattere) non spezzano il blocco (non vanno a capo) e si includono liberamente l'uno dentro all'altro. Non esistono vincoli di contenimento.

Si dividono in elementi *fontstyle* e elementi *phrase*.

I tag *fontstyle* forniscono informazioni specifiche di rendering. Molti sono deprecati, e si suggerisce comunque sempre di usare gli stili CSS.

- TT (TeleType, font monospaziato, ad es. Courier),
- I (corsivo),
- B grassetto,
- U (sottolineato - deprecato),
- S e STRIKE (testo barrato - deprecato),
- BIG, SMALL (testo più grande e più piccolo)



Elementi inline (2)

```
<body class="container-fluid">
  <h1>Inline elements</h1>
  <p>Fontstyle elements define specific rendering. </p>
  <ul>
    <li>text in <i>italics</i>.</li>
    <li>text in <b>bold</b>.</li>
    <li>text in <tt>teletype</tt> (i.e., monospaced font).</li>
    <li>text in <u>underlined</u>.</li>
    <li>text in <s>strike-through</s> and also <strike>strike-through</strike>.</li>
    <li>text in <sup>superscript</sup> and <sub>subscript</sub>.</li>
    <li>text in <big>bigger</big> and <small>smaller</small>.</li>
  </ul>
  <p>The phrase elements define specific meanings.</p>
  <ul>
    <li><em>emphasis</em></li>
    <li><strong>major emphasis</strong></li>
    <li><dfn>definition</dfn></li>
    <li><code>program fragment</code></li>
    <li><kbd>text entered by the user</kbd></li>
    <li><var>program variable</var></li>
    <li><abbr>abbreviation</abbr> and <acronym>acronym</acronym></li>
    <li><span>generic inline element</span></li>
    <li><cite>short quote</cite></li>
    <li><q>long quote</q>.</li>
  </ul>
</body>
```

Elementi inline (2)

```
<body class="container-fluid">
  <h1>Inline elements</h1>
  <p>Fontstyle elements define specific re
  <ul>
    <li>text in <i>italics</i>.</li>
    <li>text in <b>bold</b>.</li>
    <li>text in <tt>teletype</tt> (i.e.,
    <li>text in <u>underlined</u>.</li>
    <li>text in <s>strike-through</s> an
    <li>text in <sup>superscript</sup> a
    <li>text in <big>bigger</big> and <s
  </ul>
  <p>The phrase elements define specific m
  <ul>
    <li><em>emphasis</em></li>
    <li><strong>major emphasis</strong><
    <li><dfn>definition</dfn></li>
    <li><code>program fragment</code></l
    <li><kbd>text entered by the user</k
    <li><var>program variable</var></li>
    <li><abbr>abbreviation</abbr> and <a
    <li><span>generic inline element</sp
    <li><cite>short quote</cite></li>
    <li><q>long quote</q>.</li>
  </ul>
</body>
```

Fontstyle elements define specific rendering.

- text in *italics*.
- text in **bold**.
- text in **teletype** (i.e., monospaced font).
- text in underlined.
- text in ~~strike-through~~ and also ~~strike-through~~.
- text in ^{superscript} and subscript.
- text in **bigger** and smaller.

The phrase elements define specific meanings.

- *emphasis*
- **major emphasis**
- *definition*
- **program fragment**
- **text entered by the user**
- *program variable*
- abbreviation and acronym
- generic inline element
- *short quote*
- “long quote”.

Elementi inline (2)

```
<body class="container-fluid">
  <h1>Inline elements</h1>
  <p>Fontstyle elements define specific rendering.
  <ul>
    <li>text in <i>italics</i>.</li>
    <li>text in <b>bold</b>.</li>
    <li>text in <tt>teletype</tt> (i.e., monospaced font).</li>
    <li>text in <u>underlined</u>.</li>
    <li>text in <s>strike-through</s> and <del>strike-through</del>.</li>
    <li>text in <sup>superscript</sup> and <sub>subscript</sub>.</li>
    <li>text in <big>bigger</big> and <small>smaller</small>.</li>
  </ul>
  <p>The phrase elements define specific meanings.
  <ul>
    <li><em>emphasis</em></li>
    <li><strong>major emphasis</strong></li>
    <li><dfn>definition</dfn></li>
    <li><code>program fragment</code></li>
    <li><kbd>text entered by the user</kbd></li>
    <li><var>program variable</var></li>
    <li><abbr>abbreviation</abbr> and <acronym>acronym</acronym></li>
    <li><span>generic inline element</span></li>
    <li><cite>short quote</cite></li>
    <li><q>long quote</q>.</li>
  </ul>
</body>
```

Fontstyle elements define specific rendering.

- text in *italics*.
- text in **bold**.
- text in `teletype` (i.e., monospaced font).
- text in underlined.
- text in ~~strike-through~~ and also ~~strike-through~~.
- text in ^{superscript} and _{subscript}.
- text in **bigger** and smaller.

The phrase elements define specific meanings.

- *emphasis*
- **major emphasis**
- *definition*
- `program fragment`
- **text entered by the user**
- *program variable*
- abbreviation and acronym
- generic inline element
- short quote
- "long quote".

Elementi di blocco

I tag di blocco definiscono l'esistenza di blocchi di testo che contengono elementi inline.

Elementi fondamentali:

- `<p>`: paragrafo,
- `<div>`: Blocco generico,
- `<pre>`: blocco preformattato,
- `<address>`: l'autore della pagina,
- `<blockquote>`: Citazione lunga

Blocchi con ruolo strutturale

- `<h1>`, `<h2>`, `<h3>`, `<h4>`, `<h5>`, `<h6>`: header



Elementi blocco (2)

```
<body class="container-fluid">
  <h1>Block elements</h1>
  <p>The block elements define text blocks that contain inline elements.</p>
  <h2>Basic elements: </h2>
    <p>paragraph. White space
    is          <b>not</b> meaningful</p>
    <div>generic block</div>
    <pre>preformatted block. White space
    is          meaningful</pre>
    <address>the author of the page</address>
    <blockquote>This is a long quote</blockquote>
  <h2>Blocks with a structural role</h2>
    <h1>First level heading</h1>
    <h2>Second level heading</h2>
    <h3>Third level heading</h3>
    <h4>Fourth level heading</h4>
    <h5>Fifth level heading</h5>
    <h6>Sixth level heading</h6>
</body>
```

Elementi blocco (2)

```
<body class="container-fluid">
  <h1>Block elements</h1>
  <p>The block elements define text blocks</p>
  <h2>Basic elements: </h2>
    <p>paragraph. White space
    is          <b>not</b> meaningful
    <div>generic block</div>
    <pre>preformatted block. White space
    is          meaningful</pre>
    <address>the author of the page</address>
    <blockquote>This is a long quote</blockquote>
  <h2>Blocks with a structural role</h2>
    <h1>First level heading</h1>
    <h2>Second level heading</h2>
    <h3>Third level heading</h3>
    <h4>Fourth level heading</h4>
    <h5>Fifth level heading</h5>
    <h6>Sixth level heading</h6>
  </body>
```

Block elements

The block elements define text blocks that contain inline elements.

Basic elements:

paragraph. White space is **not** meaningful

generic block

preformatted block. White space
is meaningful

the author of the page

This is a long quote

Blocks with a structural role

First level heading

Second level heading

Third level heading

Fourth level heading

Fifth level heading

Sixth level heading

Elementi blocco (2)

```
<body class="container-fluid">
  <h1>Block elements</h1>
  <p>The block elements define text
  <h2>Basic elements: </h2>
    <p>paragraph. White space
    is          <b>not</b> meanin
    <div>generic block</div>
    <pre>preformatted block. Whit
    is          meaningful</pre>
    <address>the author of the p
    <blockquote>This is a long q
  <h2>Blocks with a structural role
    <h1>First level heading</h1>
    <h2>Second level heading</h2>
    <h3>Third level heading</h3>
    <h4>Fourth level heading</h4>
    <h5>Fifth level heading</h5>
    <h6>Sixth level heading</h6>
</body>
```

Block elements

The block elements define text blocks that contain inline elements.

Basic elements:

paragraph. White space is **not** meaningful

generic block

```
preformatted block. White space
                        is          meaningful
```

the author of the page

This is a long quote

Blocks with a structural role

First level heading

Second level heading

Third level heading

Fourth level heading

Fifth level heading

Sixth level heading

Elementi di lista

Le liste sono contenitori di elementi omogenei

Liste più importanti sono:

- `<ul>`: Unordered list: lista non ordinata di elementi `<li>`
 - Attributi: `type` (`disc`, `square`, `circle`)
- `<ol>`: ordered list: lista ordinata di elementi `<li>`
 - Attributi: `start` (valore iniziale) e `type` (`1`, `a`, `A`, `i`, `I`).
- `<dl>`: lista di definizioni, composta di molti elementi `<dt>` (definition term) e `<dd>` (definition data)



Elementi di lista (2)

```
<body class="container-fluid">
  <h1>List elements</h1>
  <ul>
    <li>First element</li>
    <li>Second element</li>
    <li>Third element</li>
  </ul>
  <ol>
    <li>First element</li>
    <li>Second element</li>
    <li>Third element</li>
  </ol>
  <dl>
    <dt>First term</dt><dd>First definition</dd>
    <dt>Second term</dt><dd>Second definition</dd>
    <dt>Third term</dt><dd>Third definition</dd>
  </dl>
</body>
```

Elementi di lista (2)

```
<body class="container-fluid">
  <h1>List elements</h1>
  <ul>
    <li>First element</li>
    <li>Second element</li>
    <li>Third element</li>
  </ul>
  <ol>
    <li>First element</li>
    <li>Second element</li>
    <li>Third element</li>
  </ol>
  <dl>
    <dt>First term</dt><dd>First definition
    <dt>Second term</dt><dd>Second definition
    <dt>Third term</dt><dd>Third definition
  </dl>
</body>
```

List elements

- First element
- Second element
- Third element

1. First element
2. Second element
3. Third element

First term

First definition

Second term

Second definition

Third term

Third definition

Elementi di lista (2)

```
<body class="container-fluid">
  <h1>List elements</h1>
  <ul>
    <li>First element</li>
    <li>Second element</li>
    <li>Third element</li>
  </ul>
  <ol>
    <li>First element</li>
    <li>Second element</li>
    <li>Third element</li>
  </ol>
  <dl>
    <dt>First term</dt><dd>First definition</dd>
    <dt>Second term</dt><dd>Second definition</dd>
    <dt>Third term</dt><dd>Third definition</dd>
  </dl>
</body>
```

List elements

- First element
- Second element
- Third element

1. First element
2. Second element
3. Third element

First term

First definition

Second term

Second definition

Third term

Third definition

Elementi di lista (3)

```
<body class="container-fluid">
  <h1>List elements</h1>
  <ul>
    <li>First element</li>
    <li type="square">Second element</li>
    <li type="circle">Third element</li>
  </ul>
  <ol>
    <li type="1">First element</li>
    <li type="i">Second element</li>
    <li type="A">Third element</li>
  </ol>
</body>
```



Elementi di lista (3)

```
<body class="container-fluid">
  <h1>List elements</h1>
  <ul>
    <li>First element</li>
    <li type="square">Second element</li>
    <li type="circle">Third element</li>
  </ul>
  <ol>
    <li type="1">First element</li>
    <li type="i">Second element</li>
    <li type="A">Third element</li>
  </ol>
</body>
```

List elements

- First element
- Second element
- Third element

1. First element
- ii. Second element
- C. Third element

Elementi generici

`<div>` e `<span>` sono cosiddetti *elementi generici*: privi di caratteristiche semantiche o presentazionali predefinite, assumono quelle desiderate con l'aiuto dei loro attributi (`style`, `class`, `lang`).

`<div>` mantiene la natura di elemento blocco, e `<span>` la natura di elemento inline, ma ogni altra caratteristica è neutra.

Es.: identificare con elementi appositi tutti i titoli di film può realizzarsi con `<titolofilm>`, ma si esce da HTML, oppure con `<span class="titolofilm">`.



Piccoli effetti grafici

<hr> (horizontal rule): una piccola riga orizzontale attraverso lo schermo. Si può controllarne la larghezza e lo spessore.

**
** (break): una andata a capo forzata (non spezza logicamente il paragrafo, impone semplicemente il cambio riga)



Elementi di struttura

HTML 4 fornisce pochi elementi per strutturare i contenuti in sezioni e sottosezioni: il contenitore gerarchico `<div>` (cui associare un ruolo tramite l'attributo `@class`) e gli elementi `<h1>, ..., <h6>` (che però non danno “profondità” al documento e devono essere usati in uno specifico ordine)

HTML 5 introduce nuovi elementi con una semantica precisa, per organizzare il documento in contenitori che possono essere più facilmente manipolati, esportati, modificati, etc.:

`<main>`: la parte principale della pagina, ovvero un contenitore di testo o di sottosezioni che rappresenta la parte più importante della pagina. Al suo interno possiamo trovare:

- `<section>`: un contenitore generico annidabile.
- `<article>`: una parte del documento *self-contained* e pensata per essere ri-usata e distribuita come unità atomica (post di un blog, news, feed, etc.)

`<aside>`: una sezione collegata al testo ma separata dal flusso principale (note a margine, sidebar, incisi, pubblicità, etc.)

`<header>` e `<footer>`: elementi iniziali e finali di un documento

`<nav>`: liste di navigazione



HTML 4 corretto ma...

```
<h1>Benvenuto nella mia home!</h1>
```

```
<p>Si divide in due parti: short-bio e post  
giornalieri...</p>
```

```
<h2>Bio</h2>
```

```
<p>Nato a, laureato il, dot
```

Benvenuto nella mia home!

```
<h2>9 marzo 2017</h2>
```

```
<p>Che giornata grigia.</p>
```

Si divide in due parti: short-bio e post giornalieri...

```
<h2>13 marzo 2017</h2>
```

Bio

```
<p>Una bella giornata di so
```

Nato a, laureato il, dottorato il, etc.

9 marzo 2017

Che giornata grigia.

13 marzo 2017

Una bella giornata di sole.



...molto meglio (HTML 4)

```
<div id="main">
  <h1>Benvenuto nella mia home!</h1>
  <p>Si divide in due parti: short-bio e post
    giornalieri...</p>

  <div id="bio">
    <h1>Bio</h1>
    <p>Nato a, laureato il, dottorato il, etc.</p>
  </div>

  <div id="posts">
    <div id="09-03-17">
      <h1>09 marzo 2017</h1>
      <p>Che giornata grigia.</p>
    </div>

    <div id="13-03-17">
      <h1>13 marzo 2017</h1>
      <p>Una bella giornata di sole.</p>
    </div>
  </div>
</div>
```



E in HTML 5...

```
<main>
  <h1>Benvenuto nella mia home!</h1>
  <p>Si divide in due parti: short-bio e post
    giornalieri...</p>

  <section id="bio">
    <h1>Bio</h1>
    <p>Nato a, laureato il, dottorato il, etc.</p>
  </section>

  <section id="posts">
    <article id="09-03-17">
      <h1>9 marzo 2017</h1>
      <p>Che giornata grigia.</p>
    </article>

    <article id="13-03-17">
      <h1>13 marzo 2017</h1>
      <p>Una bella giornata di sole.</p>
    </article>
  </section>
</main>
```



Header e footer

HTML 5 definisce inoltre due specifici elementi usati sia per contenuti ripetuti (nel caso di paginazione) e sia per contenuti che solitamente sono visualizzati all'inizio o alla fine di una sezione, sottosezione, articolo, etc.

<header>: contiene l'intestazione della sezione corrente o dell'intera pagina ed è solitamente usato per tabelle di contenuti, indici, form di ricerca, intestazioni.

- Può essere anche usato come contenitore degli headings H1, .., H6.

<footer>: contiene la “parte conclusiva” della sezione corrente o dell'intera pagina. E' usato principalmente per mostrare informazioni (testuali, non metadati) sugli autori della pagina, copyright, produzione, licenze

- Non deve essere visualizzato necessariamente a fondo pagina
- Può essere usato anche per contenere intere sezioni come appendici, allegati tecnici, colophon, etc.



Liste di navigazione

HTML 5 riprende da XHTML 2.0 anche le “liste di navigazione” ossia particolari sezioni dedicate a raggruppare link alla pagina corrente (o a sezioni di) o ad altre pagine

Si usa l'elemento `<nav>` molto spesso in combinazione con `<header>` e `<footer>`.

Le sezioni `<nav>` sono molto utili per l'accessibilità, in quanto possono essere più facilmente identificate e accedute anche da utenti disabili (tramite screenreaders o altri ausili)

Possono essere usate anche per attivare/disattivare le funzionalità di navigazione in base allo user-agent (browser) che sta accedendo alla pagina.



Esempio

```
<header>
  <nav>
    <h1>Navigation</h1>
    <ul>
      <li><a href="/">Home</a></li>
      <li><a href="/standards/">Standards</a></li>
      <li><a href="/participate/">Participate</a></li>
      <li><a href="/membership">Membership</a></li>
      <li><a href="/Consortium/">About W3C</a></li>
    </ul>
  </nav>
</header>
<main>... </main>
<footer>
  <nav>
    <h1>Contact W3C</h1>
    <ul>
      <li><a href="/Consortium/contact">Contact</a></li>
      <li><a href="/Help/">Help and FAQ</a></li>
      <li><a href="/Consortium/sup">Donate</a></li>
      <li><a href="/Consortium/siteindex">Site Map</a></li>
    </ul>
  </nav>
</footer>
```



Conclusioni

In questa lezione abbiamo cercato di fare chiarezza tra:

- HTML
- (X)HTML 5
- HTML 4.x
- XHTML 1.0



Riferimenti

The HTML Living Standard:

<http://www.whatwg.org/html>

W3C HTML5:

<http://www.w3.org/TR/html5/>

D. Raggett, A. Le Hors, I. Jacobs, *HTML 4.01 Specification*, W3C Recommendation 24 December 1999,

<http://www.w3.org/TR/html401>

M. Altheim et alii, *Modularization of XHTML*, W3C Recommendation 10 April 2001, W3C Recommendation 10 April 2001,

<http://www.w3.org/TR/xhtml1-modularization>

M. Altheim, S. McCarron, *XHTML™ 1.1 - Module-based XHTML*, W3C Recommendation 31 May 2001,

<http://www.w3.org/TR/xhtml11>

J. Axelsson et alii, *XHTML™ 2.0*, W3C Working Draft 31 January 2003,

<http://www.w3.org/TR/2003/WD-xhtml2-20030131>





ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Fabio Vitali

Corso di tecnologie web

Fabio.vitali@unibo.it

www.unibo.it